



湖南大学
HUNAN UNIVERSITY

第二章 高级数据结构

湖南大学计算机学院

引言



- 对于计算机而言，数组拥有最高的访问效率（无指针跳转、内存连续）。
- 本节所介绍的高级数据结构，就是如何巧妙地利用数组来模拟复杂逻辑，从而兼顾性能与功能。
- 课程目标：
 - 哈希表：如何通过数学映射，在数组中实现常数级检索？

目录

第一节

哈希表

第二节

自组织表

快速查找问题



手机上都有基本的通讯功能，有通讯功能必然存在通讯录

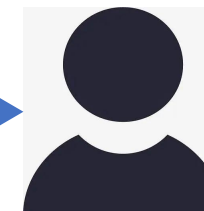
通讯录功能： 输入姓名能映射出电话号码，输入电话号码能映射出姓名



1234 5678



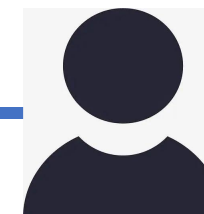
张三



8765 4321



李四



哈希表的作用



哈希表是一种基于数组实现的数据结构。它通过把关键码值映射到数组中一个位置来访问记录，存放记录的数组叫做哈希表。
这个映射函数叫做**哈希函数**。

哈希表的应用场景：

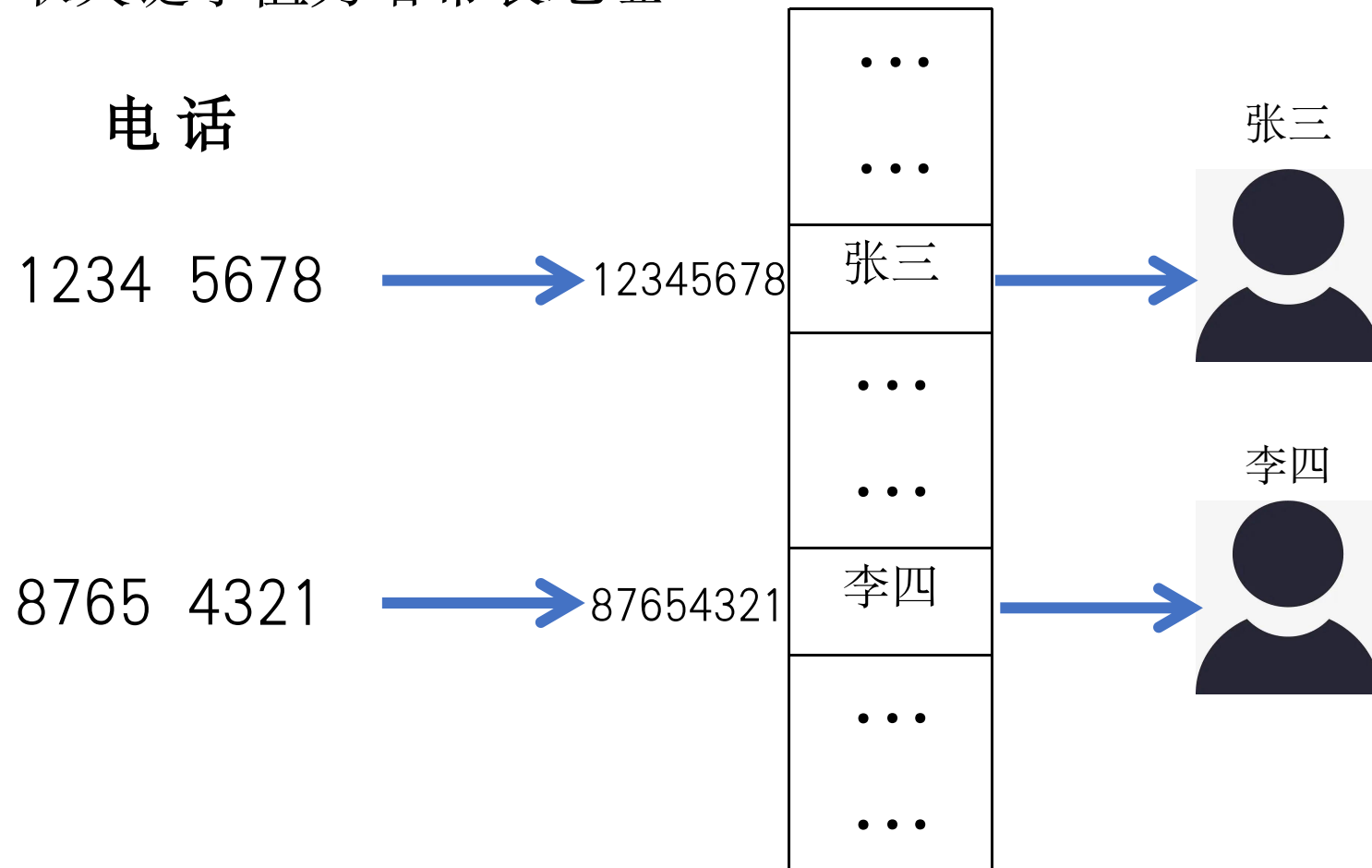
- 网页快速查找信息
- DNA匹配：字符串之间的匹配

课程目标： 哈希表定义以及常用哈希函数的定义和计算，如除法哈希

直接寻址



直接寻址：取关键字值为哈希表地址



直接寻址优缺点



优点：查找速度快

缺点：对于全域较大，但是元素却十分稀疏的情况，使用这种存储方式将浪费大量的存储空间。

哈希函数



为了克服直接寻址技术的缺点，而又保持其快速字典操作的优势，我们可以利用哈希函数 $h: U \rightarrow \{0, 1, 2, \dots, m-1\}$ 来计算关键字 k 所在的位置

哈希函数 $h(k)$ 的作用是将范围较大的关键字映射到一个范围较小的集合中。形式化来说，设关键字集 K 中有 n 个关键字，哈希表长为 m ，哈希函数 h 满足以下要求：

1. 对任意 $k \in K$ ，有 $0 \leq h(k) \leq m-1$ ；
2. 对任意 $k \in K$ ， $h(k)$ 取 $[0, m-1]$ 中任一值的概率相等，即每个值取到的概率都是 $1/m$ 。

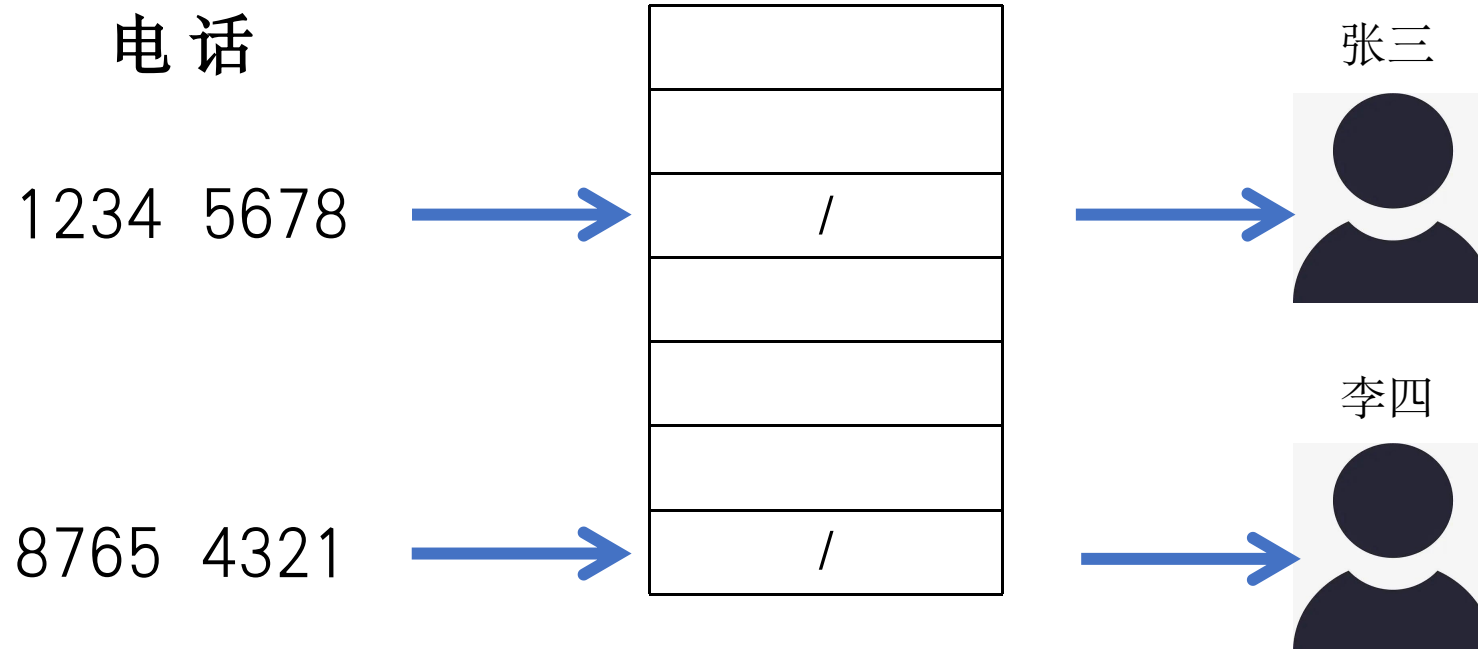
除法哈希函数



假设 m 是哈希表大小，通过取关键字值 k 除以 m 的余数，将关键字值 k 映射到哈希表中的一个位置

$$h(k) = k \bmod m$$

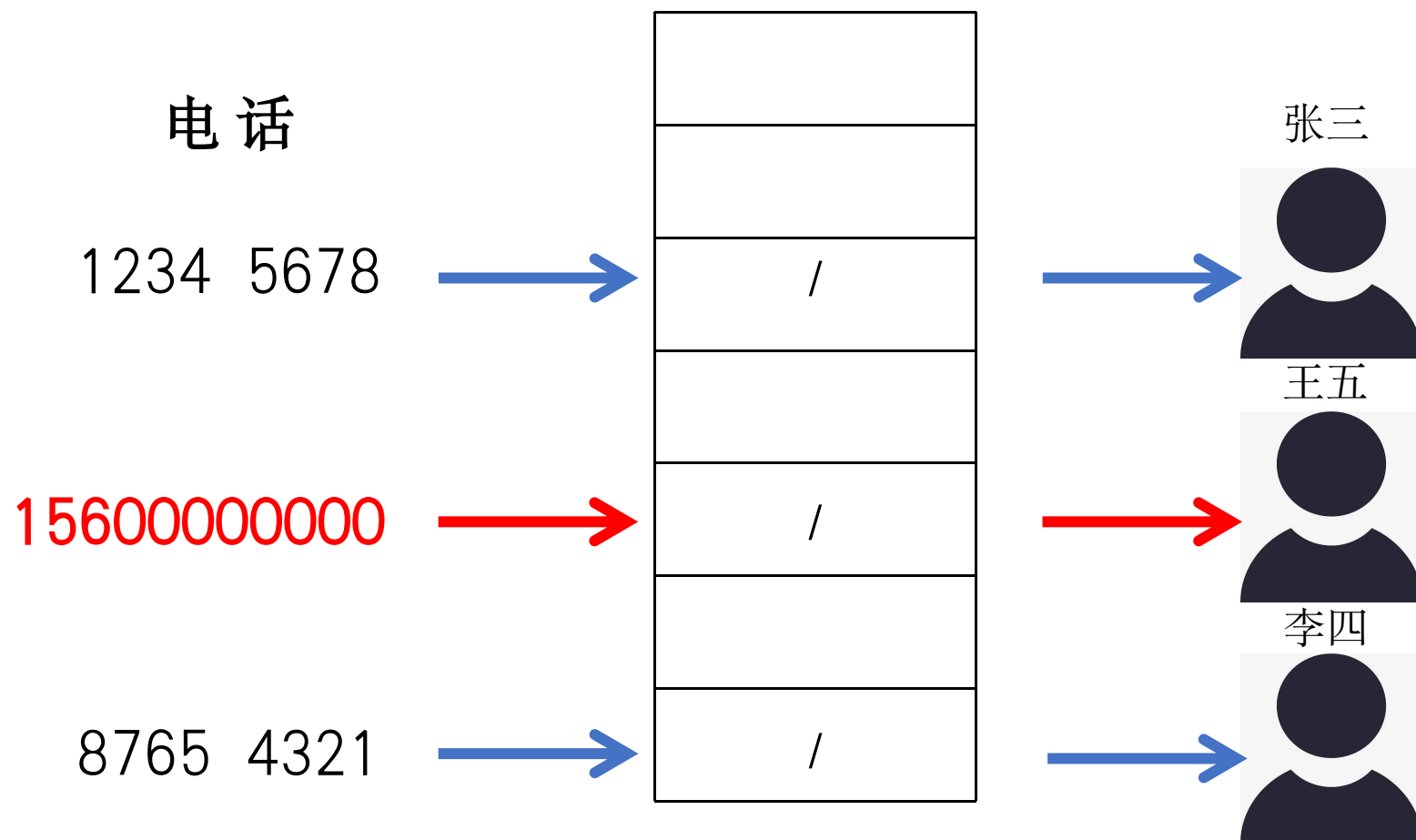
假设 $m=7$



除法哈希函数



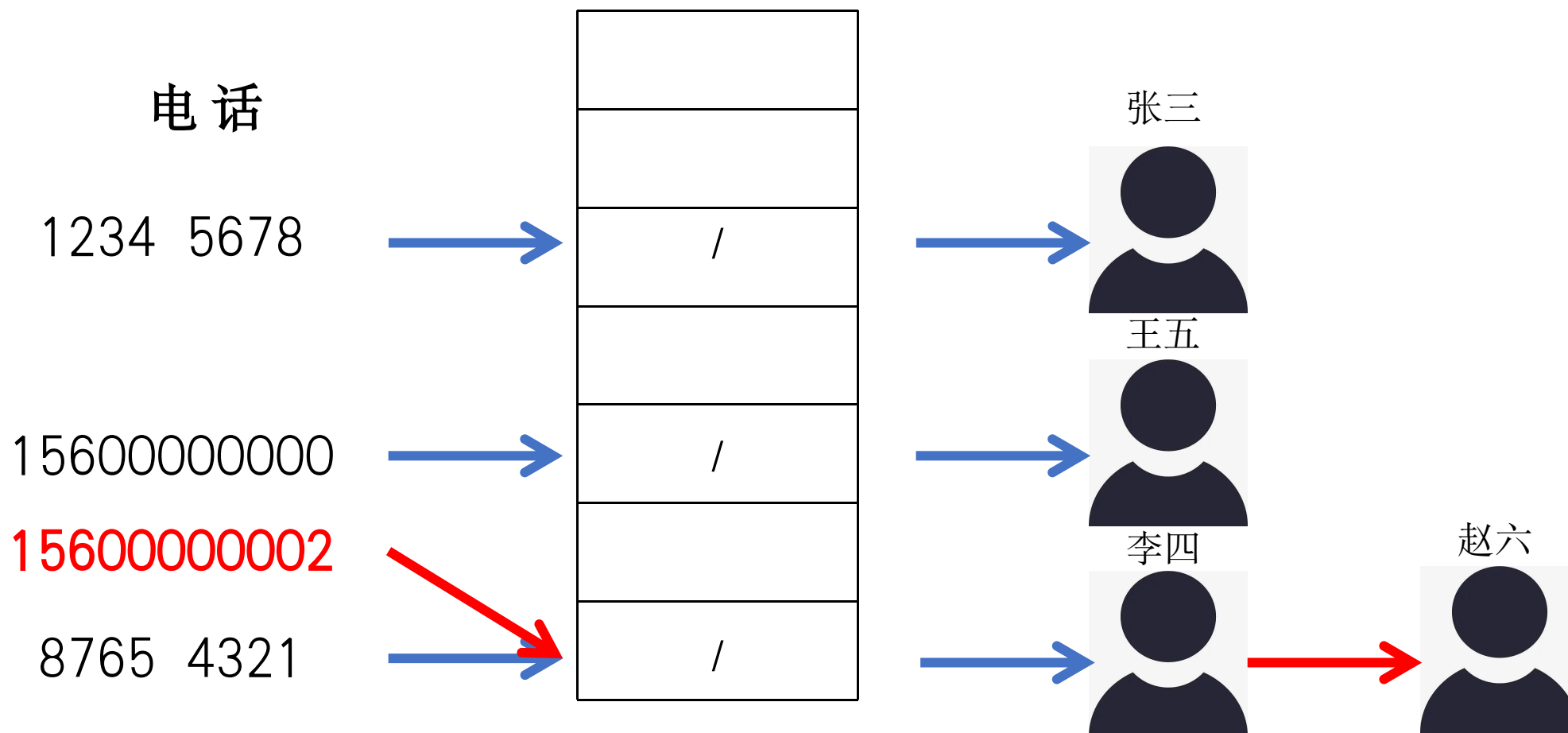
计算电话为156000000000的王五在上述哈希表中位置



哈希函数的核心挑战——冲突



计算电话为156000000002的赵六在上述哈希表中位置



除法哈希函数的不足



- 针对具有规律性的键值集合，有规律性问题：
 - 假设键值为 $x, 2x, 3x, 4x, \dots$ （例如：若 x 与 m 存在公约数 d ）
 - 若 x 与所选模数 m 有公共因子 d ，则哈希表仅能利用其 $1/d$ 的空间：
 - ✓ x 的倍数序列会循环映射，导致 $d-1$ 个槽位始终为空；
 - ✓ 举例：若 m 是 2 的幂，而所有键均为偶数 $\rightarrow$ 仅使用一半空间。
- 解决方案：令 m 为质数但寻找大质数本身计算成本较高；
 - 此外，除法运算效率低下（相比乘法或位移操作慢得多）。

乘法哈希



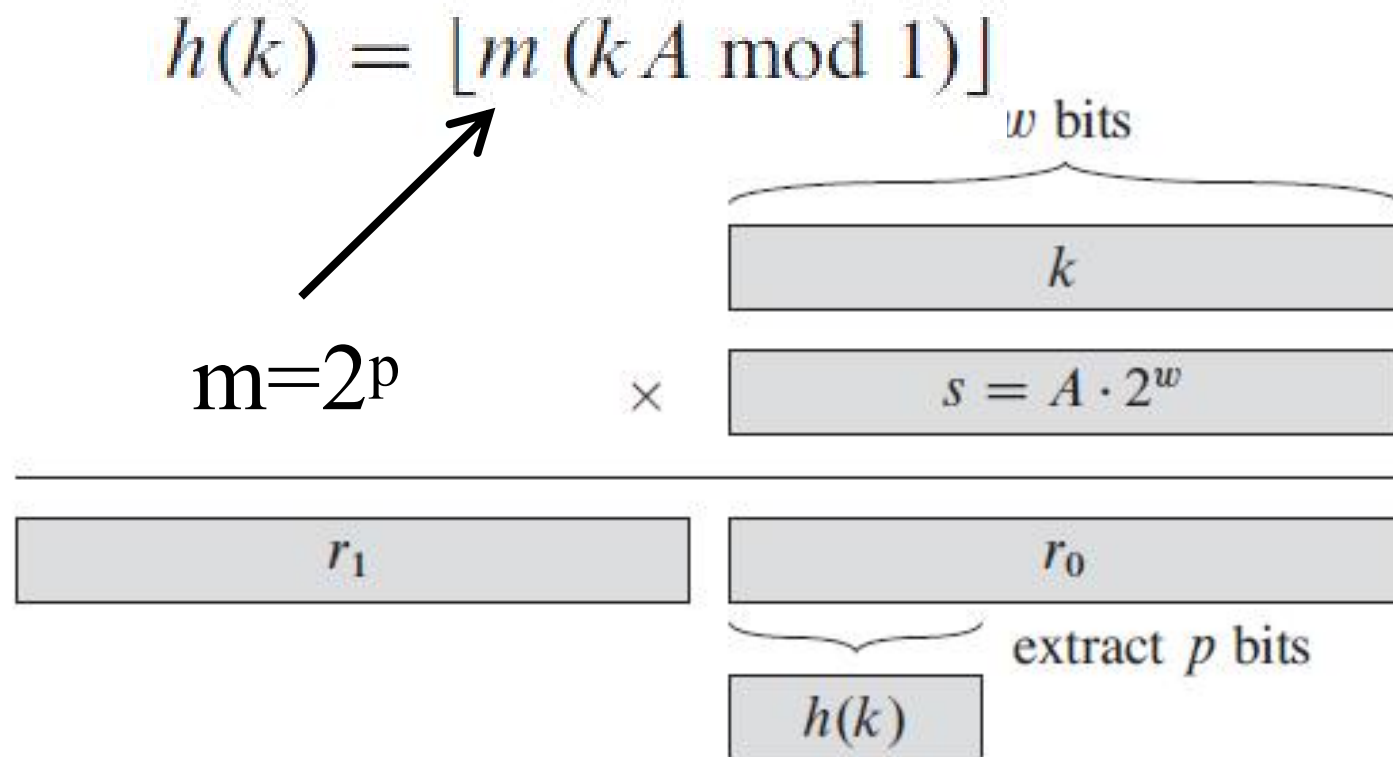
原理：将键值 k 乘以一个常数 A ($0 < A < 1$)，提取结果的小数部分，再乘以表长 m 并取整。

优点：

1. 速度快：若 $m=2^p$ ，可用位移操作替代除法。
2. 鲁棒性强： A 选得好，对任意 m 效果均佳。

缺点：

1. 涉及浮点运算（在某些嵌入式环境可能较慢）。
2. 相比除留余数法，对质数 m 的依赖性低但常数 A 需精心挑选。



$$A \approx (\sqrt{5} - 1) / 2 = 0.618033988$$

哈希函数：哈希值



Key不是数字，是字符串呢？

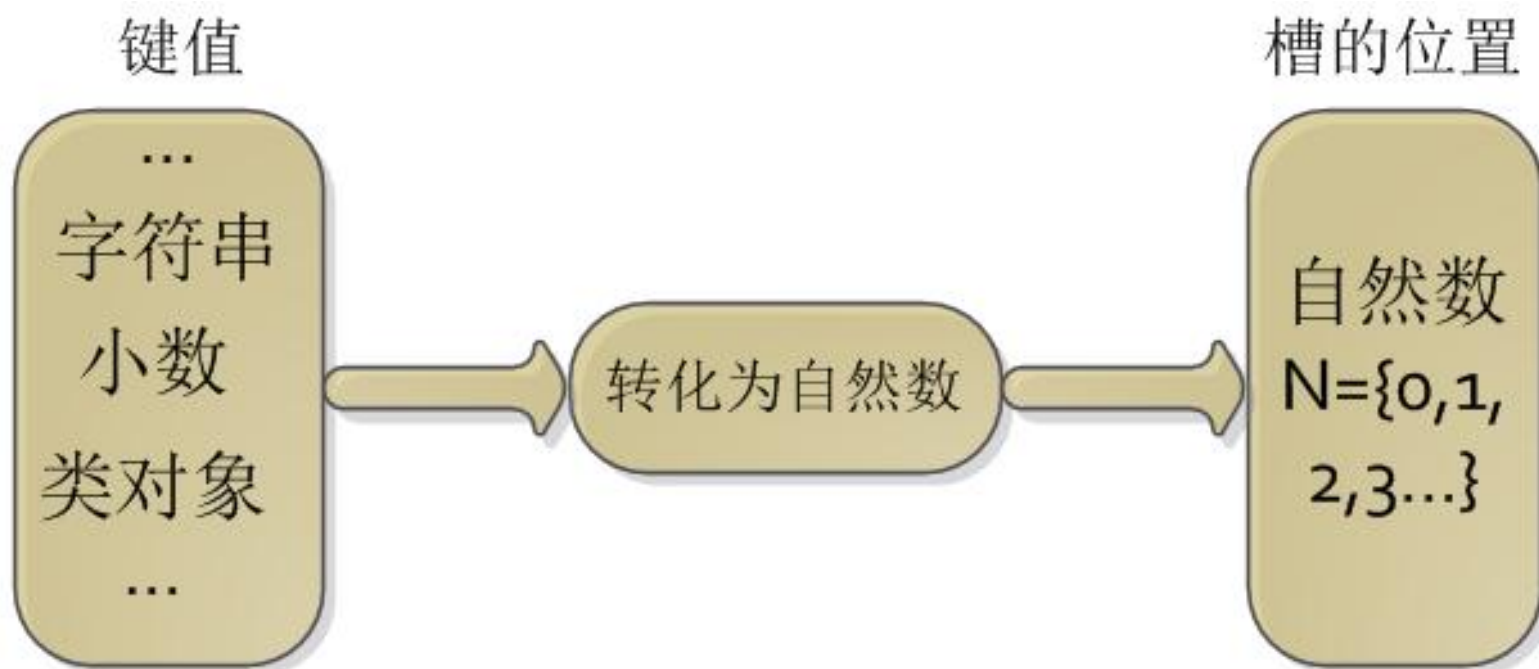
Key="abcdef"



Hash值



哈希函数:哈希值



Java: hashCode

C++: ASCII码相加

Python:

Key="abcdef"

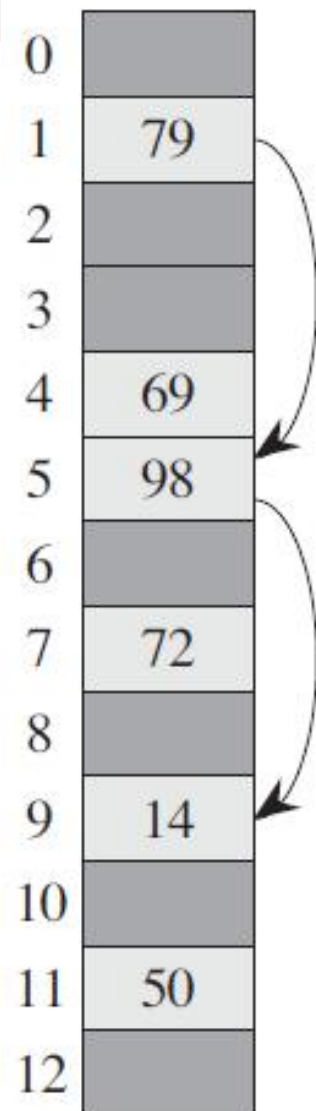
A=hash(Key)

开放寻址法



开放寻址法（open addressing）中，所有元素都存放在槽中，在链表法哈希表中，每个槽中保存的是相应链表的指针，为了维护一个链表，链表的每个结点必须有一个额外的域来保存它的前驱和后继结点。

开放寻址法



开放寻址法



三种常用技术来计算开放寻址法中的探查序列

1. 线性探查
2. 二次探查
3. 双重哈希

开放寻址法-线性探查



$$h(k, i) = (h'(k) + i) \bmod m$$

关键词 (key)	47	7	29	11	9	84	54	20	30
散列地址 $h(\text{key})$	3	7	7	0	9	7	10	9	8
冲突次数	0	0	1	0	0	3	1	3	6

开放寻址法-线性探查



关键词 (key)	47	7	29	11	9	84	54	20	30
散列地址 $h(\text{key})$	3	7	7	0	9	7	10	9	8
冲突次数	0	0	1	0	0	3	1	3	6

地址 操作	0	1	2	3	4	5	6	7	8	9	10	11	12	说明
插入47				47										无冲突
插入7				47				7						无冲突
插入29				47				7	29					$d_1 = 1$
插入11	11			47				7	29					无冲突
插入9	11			47				7	29	9				无冲突
插入84	11			47				7	29	9	84			$d_3 = 3$
插入54	11			47				7	29	9	84	54		$d_1 = 1$
插入20	11			47				7	29	9	84	54	20	$d_3 = 3$
插入30	11	30		47				7	29	9	84	54	20	$d_6 = 6$

注意“聚集”现象

http://blog.csdn.net/qq_30091945

开放寻址法-二次探查



$$h(k, i) = (h'(k) + c_1 \times i + c_2 \times i^2) \bmod m$$

开放寻址法-双重哈希



$$h(k, i) = (h_1(k) + i \times h_2(k)) \bmod m$$

Cuckoo Hash 布谷鸟哈希



定义：一种解决 hash 冲突的方法，其目的是使用简单的 hash 函数来提高 hash table 的利用率，同时保证 $O(1)$ 的查询时间。基本思想是使用2个 hash 函数来处理碰撞，从而每个 key 都对应到2个位置。

Cuckoo Hash 布谷鸟哈希



操作:

- 1) 对 key 值 hash, 生成两个 hash key 值, hashk_1 和 hashk_2 , 如果对应的两个位置上有一个为空, 那么直接把 key 插入即可。
- 2) 否则, 任选一个位置, 把 key 值插入, 把已经在那个位置的 key 值踢出来。
- 3) 被踢出来的 key 值, 需要重新插入, 直到没有 key 被踢出为止。

Cuckoo Hash 布谷鸟哈希



衍生背景:

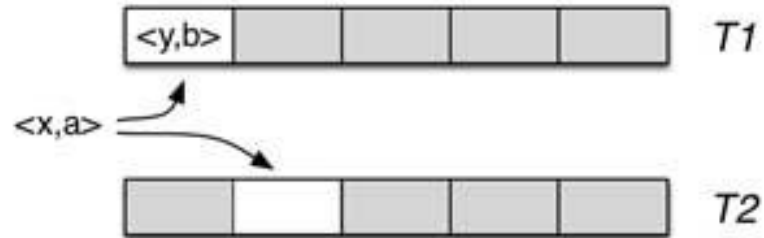
Cuckoo中文名叫布谷鸟，这种鸟有一种即狡猾又贪婪的习性，它不肯自己筑巢，而是把蛋下到别的鸟巢里，而且它的幼鸟又会比别的鸟早出生，布谷幼鸟天生有一种残忍的动作，幼鸟会拼命把未出生的其它鸟蛋挤出窝巢，今后以便独享“养父 母”的食物。借助生物学上这一典故，cuckoo hashing处理碰撞的方法，就是把原来占用位置的这个元素踢走，不过被踢出去的元素还要比鸟蛋幸运，因为它还有一个备用位置可以安置，如果备用位置上 还有人，再把它踢走，如此往复。直到被踢的次数达到一个上限，才确认哈希表已满，并执行rehash操作。

Cuckoo Hash 布谷鸟哈希



Insertion when one of the two buckets is empty

Step 1: Both buckets for $\langle x, a \rangle$ are tested, the one in T2 is empty.

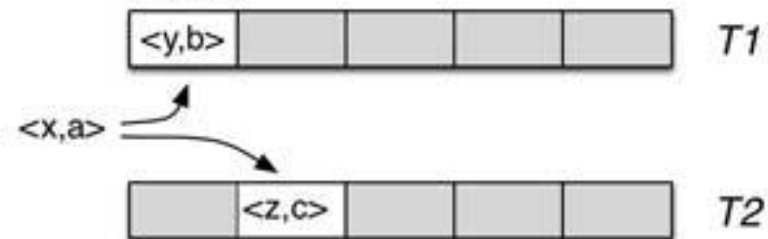


Step 2: $\langle x, a \rangle$ is stored in the empty bucket in T2.



Insertion when the two buckets already contain entries

Step 1: Here $\langle y, b \rangle$ will be withdrawn from T1 so that $\langle x, a \rangle$ can be stored.



Step 2: After $\langle x, a \rangle$ has been stored in T1, $\langle y, b \rangle$ needs to be moved to T2. The bucket in T2 may already contain an entry, if so this entry will need to be moved.



Cuckoo Hash 布谷鸟哈希



其他:

- 1) Cuckoo hash 有两种变形。一种通过增加哈希函数进一步提高空间利用率；另一种是增加哈希表，每个哈希函数对应一个哈希表，每次选择多个表中空余位置进行放置。三个哈希表可以达到80%的空间利用率。
- 2) Cuckoo hash 的过程可能因为反复踢出无限循环下去，这时候就需要进行一次循环踢出的限制，超过限制则认为需要添加新的哈希函数。

哈希的扩展应用



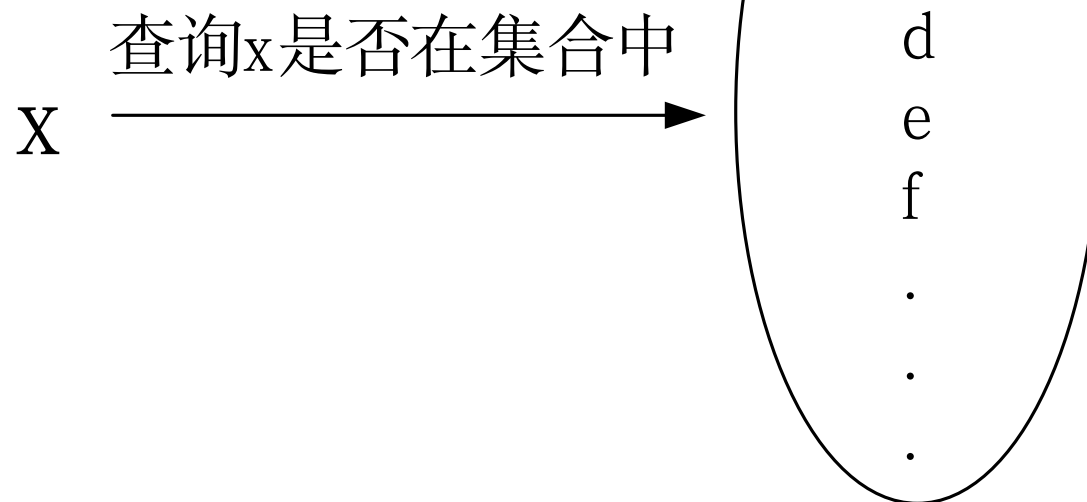
- 布鲁姆过滤器 (BloomFilter)
- 最小哈希
- 区块链

布鲁姆过滤器 (bloomfilter)



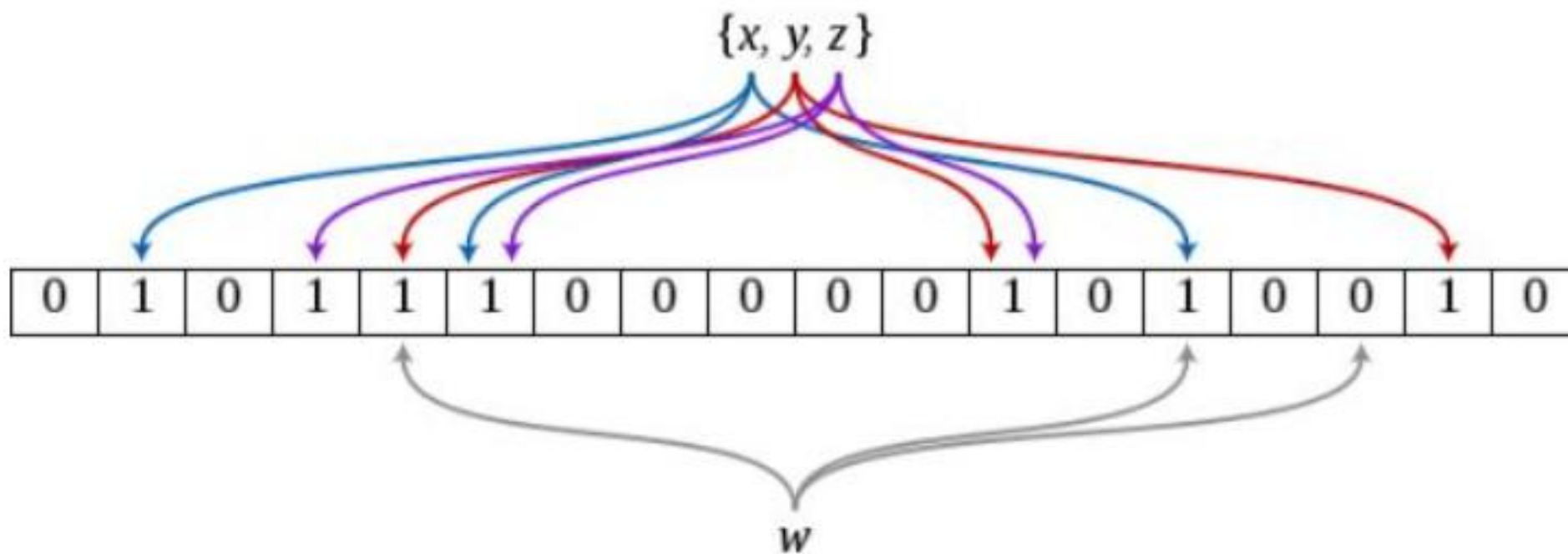
问题1

数据集



10亿数据

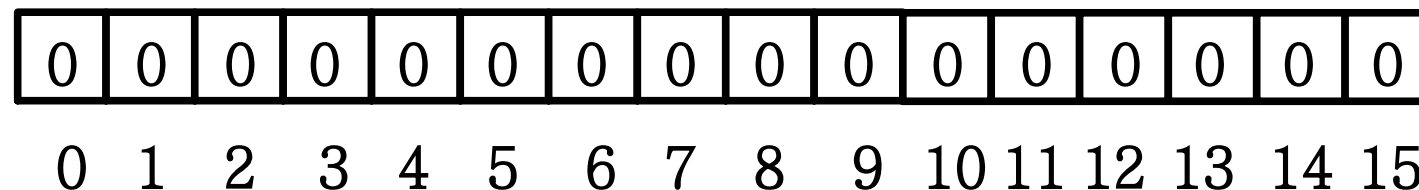
布鲁姆过滤器 (bloomfilter)



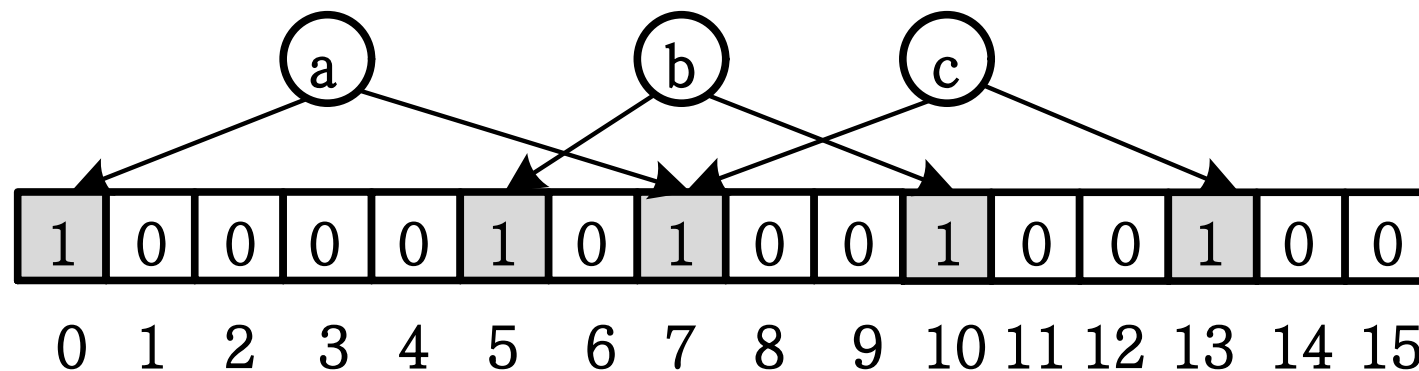
布鲁姆过滤器 (bloomfilter)



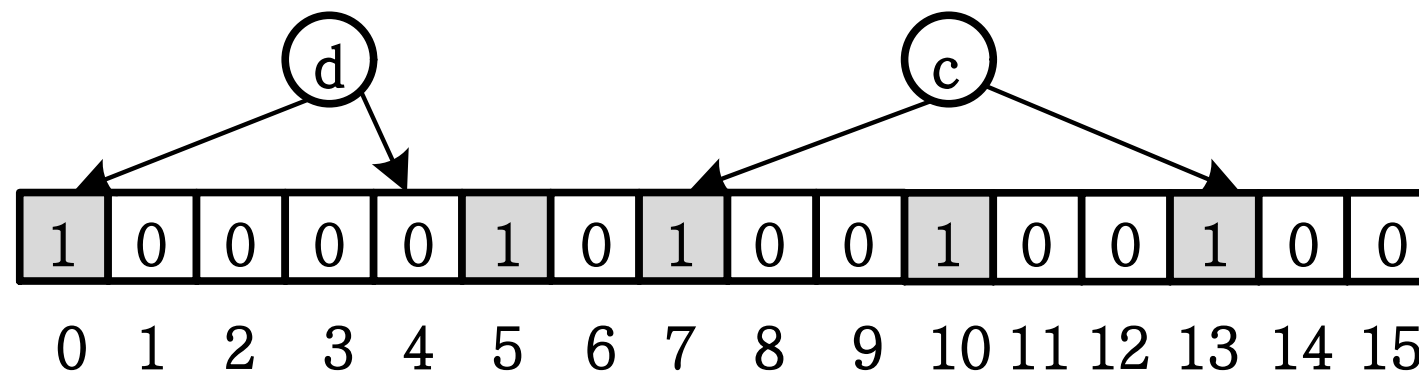
初始化



插入a, b, c



查询d, c



布鲁姆过滤器 (bloomfilter)



布鲁姆过滤器算法关键指标

- 空间复杂度
- 时间复杂度
- 误判率 (假阳性)

将不属于集合的元素误判断成属于集合中的这种假阳性误判称为一次误判。发生误判的概率称为误判率。

布鲁姆过滤器 (bloomfilter)



误判率分析 (假阳性)

理论分析, 假设 k 个哈希函数、 m 位bitset, n 个元素之后

$$f_{\text{BF}}(m, k, n) = (1 - e^{-k \cdot n / m})^k$$

布鲁姆过滤器 (bloomfilter)



误判率分析 (假阳性)

最优分析

$$k = \lceil \ln 2 (m / n) \rceil$$

$$k_{\min} = (\ln 2) \left(\frac{m}{n} \right)$$

布鲁姆过滤器 (bloomfilter)



误判率分析 (假阳性)

(1) 理论分析, 假设 k 个哈希函数、 m 位bitset, n 个元素之后

$$f_{\text{BF}}(m, k, n) = (1 - e^{-k \cdot n / m})^k$$

最小哈希



MinHash是一种局部敏感哈希技术
可以用来快速估算两个集合的相似度

Jaccard相似度



Jaccard相似度是用来计算集合相似性，也就是距离的一种度量标准

假如有集合A、B，那么 $J(A,B) = |A \cap B| / |A \cup B|$

最小哈希



$h(x)$: 把 x 映射成一个整数的哈希函数

$hmin(S)$: 集合 S 中的元素经过 $h(x)$ 哈希后, 具有最小哈希值的元素

那么对集合 A 、 B , $hmin(A) = hmin(B)$ 成立的条件是 $A \cup B$ 中具有最小哈希值的元素也在 $A \cap B$ 中

最小哈希性质



假设 $h(x)$ 是一个良好的哈希函数，它具有很好的均匀性，能够把不同元素映射成不同的整数。

所以有， $\Pr[hmin(A) = hmin(B)] = J(A,B)$ ，即集合A和B的相似度为集合A、B经过hash后最小哈希值相等的概率。

区块链典型应用——比特币



货币的特点:

1. 去中心化, 交易不经手中央银行
2. 匿名性, 交易双方之间可以互相匿名

比特币就是为了在网络上实现货币, 为此增加一个要求——不可篡改

区块链基础——加密的哈希函数



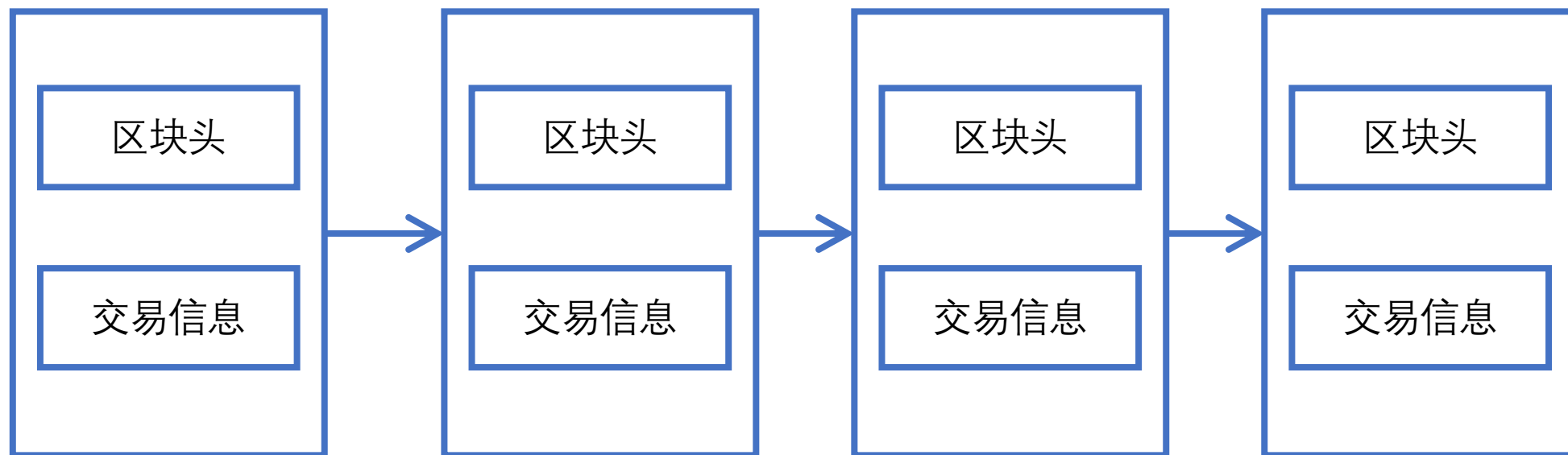
哈希函数广泛应用于信息安全，特别是保护密码。当您在网站上注册帐户时，您的密码将通过哈希函数进行转换，从而生成哈希摘要，然后由服务存储。登录后，您输入的密码将经过相同的哈希函数，并将生成的哈希值与存储的哈希值进行比较，以验证您的身份。

加密哈希函数就是实现了从哈希中破译原始密码的哈希函数。这样增强了安全性，因为即使黑客利用哈希摘要访问数据库也无法获得密码

区块链数据结构



逻辑上它是一个链式(chain)结构，每个结点上就是一个区块信息(block)，区块里面则存储了交易的信息。



区块链数据结构



区块链数据结构主要包括为区块头和区块体两部分

区块的哈希值是利用加密哈希函数对区块头中信息进行计算而得到的。

这个哈希值是区块的标识符，可以通过这个哈希值找到对应的区块，加密哈希函数保证了匿名。

当前区块中包含来前一个区块的哈希值，如此形成了一个链表。通过这个链表可以溯源整个交易来验证交易，保证了不可篡改。

比特币投票机制



比特币网络投票原理是在比特币网络中实现的一种特殊形式的网络投票。比特币网络中的投票是为了决定关键事务的变更，例如是否接受某个交易、是否添加某个区块等。

投票机制保证了去中心化

目录

第一节

哈希表

第二节

自组织表

哈希函数定义回顾



1. 开放地址方法

遇到冲突，寻找一个新的空闲哈希地址

2. 再哈希法

对于冲突的哈希值再次进行哈希处理，直至没有哈希冲突

3. 建立公共溢出区

建立公共溢出区存储所有哈希冲突的数据

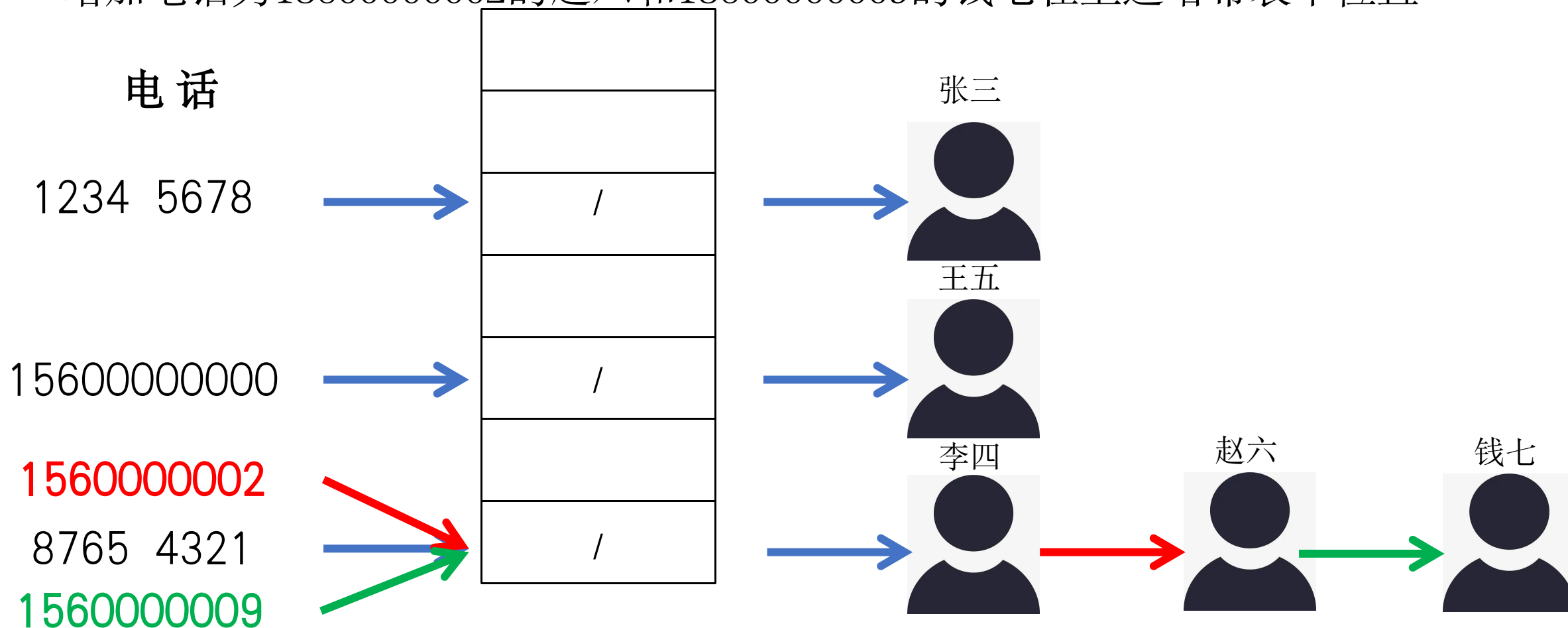
4. 链式地址法

对于相同的值，使用链表进行连接

链式地址法



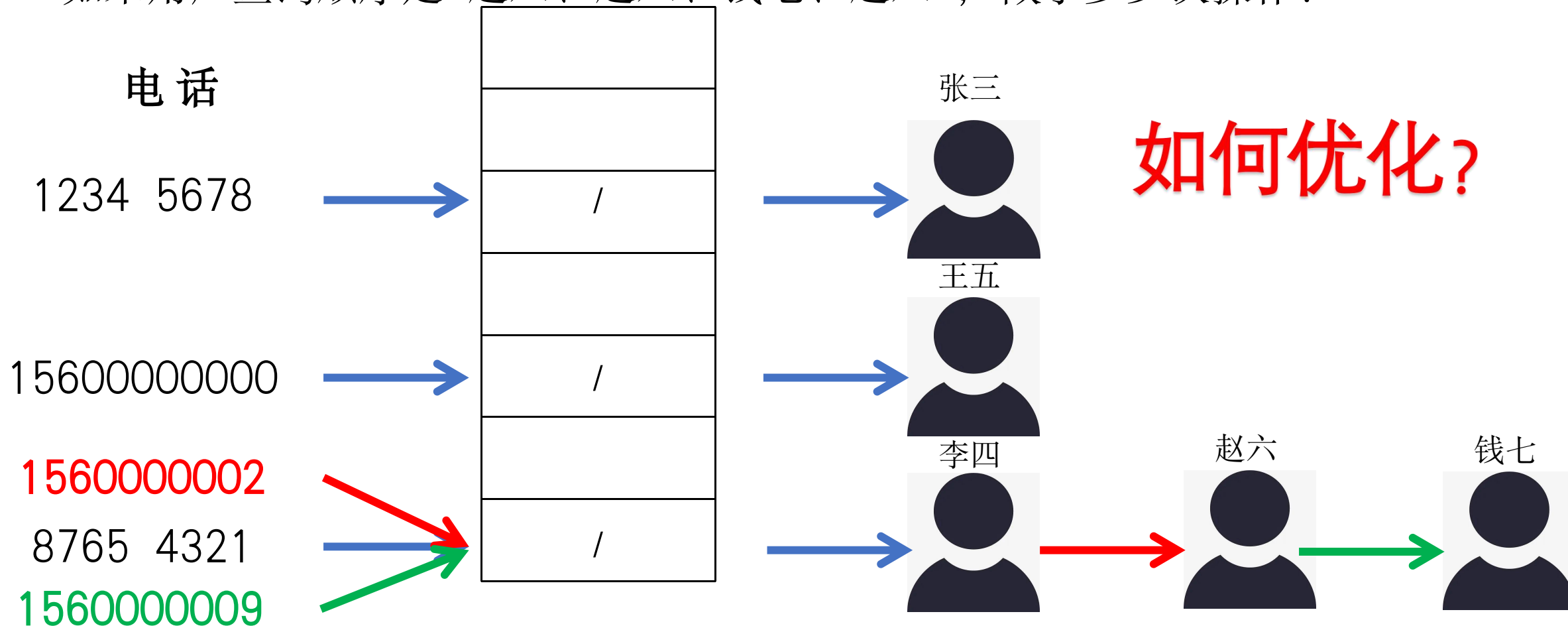
增加电话为15600000002的赵六和15600000009的钱七在上述哈希表中位置



链式地址法的代价分析



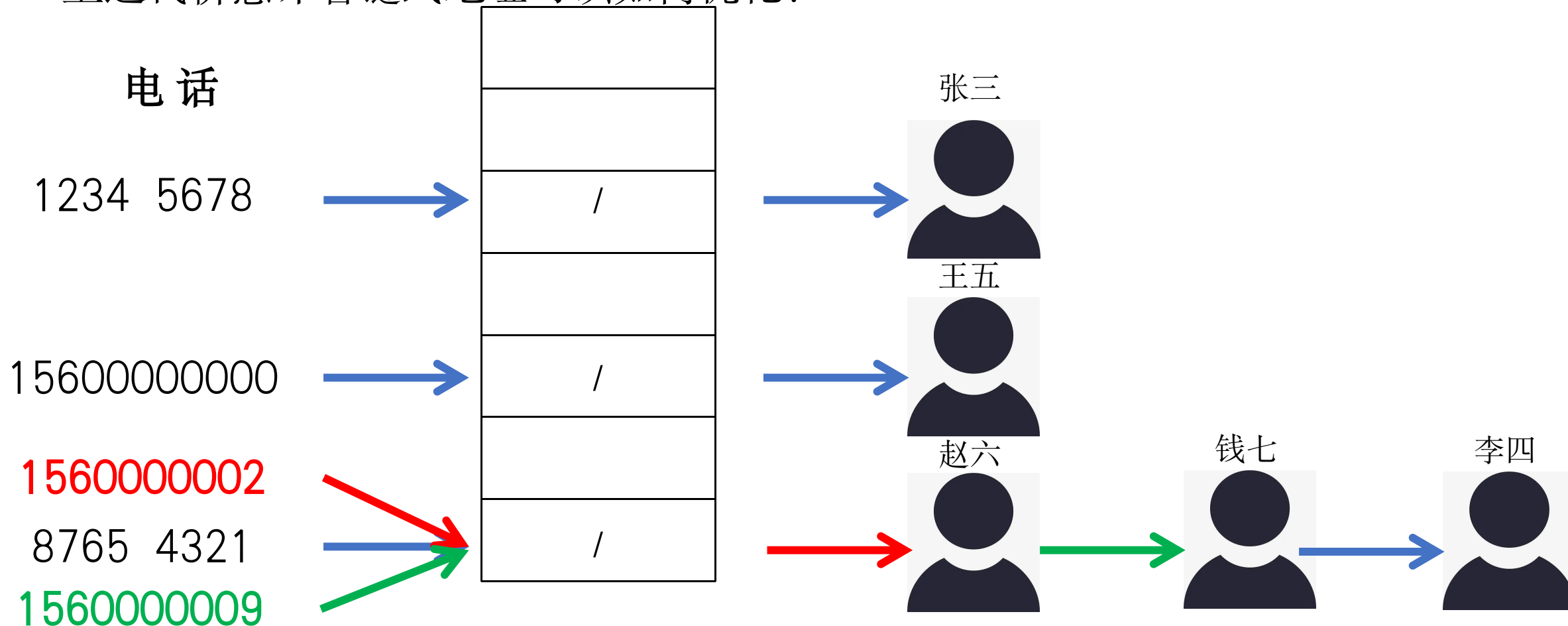
如果用户查询顺序是<赵六、赵六、钱七、赵六>，做了多少次操作？



链式地址法的代价分析



上述代价意味着链式地址可以如何优化？



定义



自组织列表是 n 个元素的链表，每个元素有一个唯一的关键字。当我们在列表中搜索元素时，需要给定一个关键字，我们搜索的是具有这个关键字的元素。

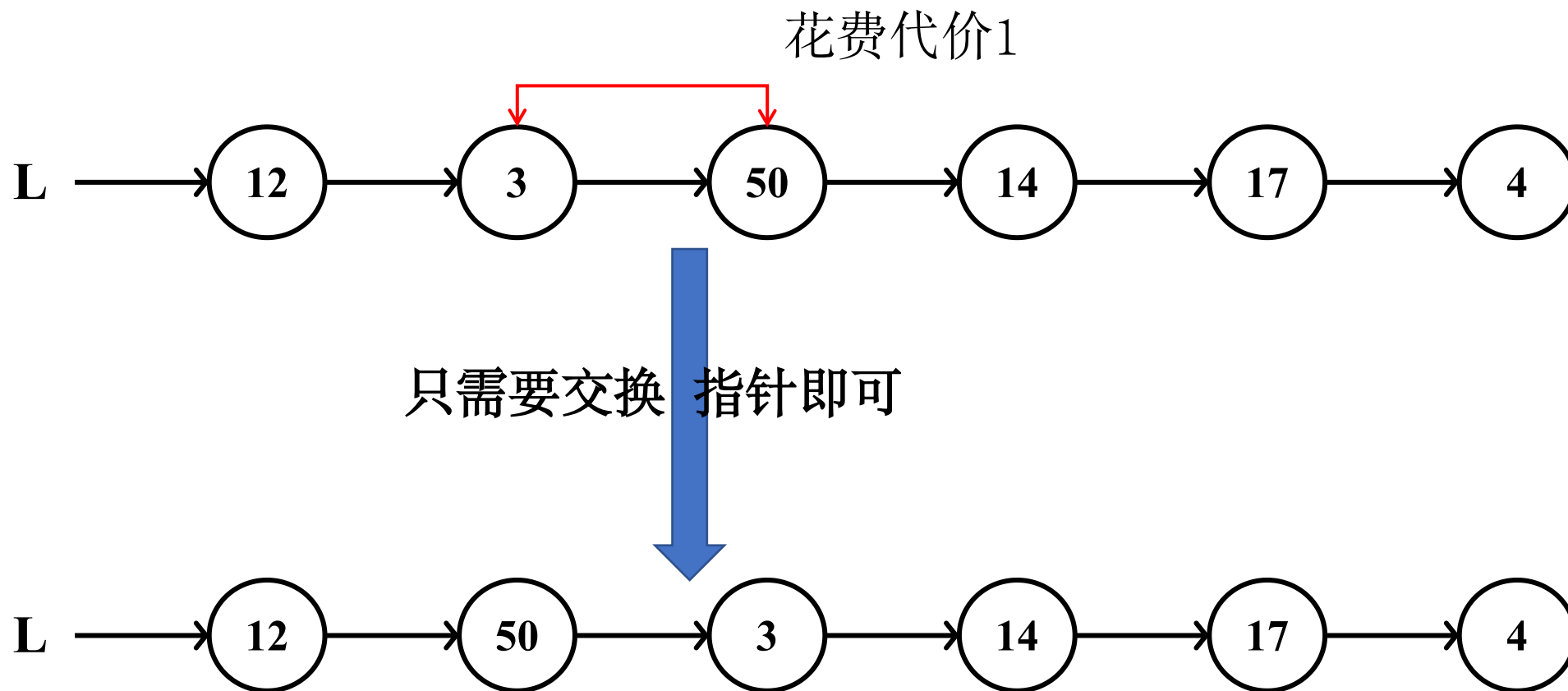
性质



一个自组织列表有两个重要性质:

- 为了在列表中查找一个元素，我们必须从表头开始遍历列表，直至遇到具有给定关键字的元素位置。如果此元素是列表的第 k 个元素，则查找代价为 k 。
- 我们可以在任意一个操作后根据给定规则重排列表元素，产生一定的代价。我们可以使用任何我们喜欢的启发式策略来决定如何重排列表。

用启发式策略重排列表

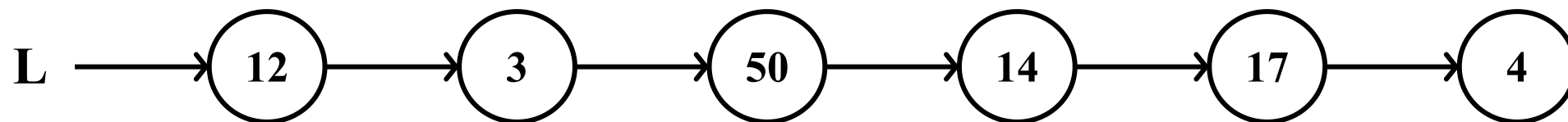


用启发式策略重排列表



假定从一个给定的 n 个元素的列表开始，并且给定了一个访问序列--关键字搜索序列 $S = \langle s_1, s_2, \dots, s_m \rangle$ 。序列的代价是序列中单个访问的代价之和。

用启发式策略重排列表



例如,我们要查找键值为14的元素,那么这次访问的代价为4。当进行一系列访问操作时,显然,某些元素会访问得更加频繁,我们想要把它放到表的前面来减少代价,所以我们进行置换操作。

在线算法



接下来我们要用“在线”操作完成。

在线算法：一个操作序列 S ，每次只发送一步的操作请求，而在线算法必须马上完成这步操作。不管接下来发生什么操作，当你考虑序列的第一项操作时，你是不考虑后面的任何一项操作的。

离线算法：可以预先看过所有的操作，提前将一个被访问次数多的元素放到表的前面。



定义： $C_A(S)$ 为用A算法计算序列S的代价。接下来讨论最坏情况下的总代价。

最坏情况下，总是要我们访问序列L的末尾元素，因为对于每个操作序列来说，都需要走到底，于是每一步的代价都为n。总代价为：

$$C_A(S) = \Omega(|S| \times n)$$

平均情况下的总代价



接下来讨论平均情况下的总代价。

假如元素 x 被访问到，它的概率是 $P(x)$ ，我们假设元素被访问的事件有一个先验分布，对于一个访问序列，某种算法的期望代价为

$$E[C_A(s)] = \sum_{x \in L} P(x) \times \text{rank}_L(x)$$

平均情况下的总代价



什么时候期望代价最小呢？

按访问概率降序排列。

它只是记录每个元素被访问的次数，并按访问次数递减的顺序维护这个表。

用移至前端启发式策略重排列表



当使用移至前端启发式策略(Move To Front, 简称MTF)时, 搜索到元素 x 后, 我们将 x 移动到列表的第一个位置(即列表的前端)。

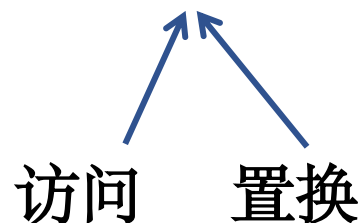
令 $\text{rank}_L(x)$ 表示元素 x 在列表 L 中的序号, 即 x 在 L 中的位置。例如, 若 x 是 L 中第4个元素, 那么 $\text{rank}_L(x) = 4$ 。令 c_i 表示用移至前端策略处理访问 s_i 的代价, 包括在列表中查找元素的代价和通过一系列相邻元素转置操作将其移至列表前端的代价。

MTF每次代价



当访问到元素 x 后，会把 x 置换到表头位置，这个代价刚好等于 x 的排位乘以2，第一部分是访问，第二部分是置换

$$Cost = 2 * \text{rank}_L(x)$$



MTF优势



实际中，数据项的访问往往呈现出一定的局部性，而非完全随机；这意味着一旦我们访问了一个特定的数据项，接下来很可能会再次访问它

正是由于这种特性，自组织列表成为了处理局部性问题的理想选择

在自组织列表下，被频繁访问的元素会逐渐向列表头部移动，从而使得下一次访问时能够迅速定位到目标元素。因此，对于具有明显局部性特征的序列S来说，采用自组织列表无疑是一种非常合适的选择。

竞争分析



我们称在线算法A是 α 竞争的，如果存在一个常数k，满足对于任意操作序列S，有

$$C_A(S) \leq \alpha \cdot C_{opt}(S) + k$$

opt代表最优算法，能对这个序列进行最好的处理。系数 α 表示在线算法A所花费的代价是最优算法代价的 α 倍。

我们称这种分析为竞争分析。

MTF策略的竞争分析



针对移至前端列表的重排问题, **MTF**代价最坏情况也不会超过任何其他启发式策略的代价的**4**倍, 即使其他启发式策略预先知道访问序列!

定理: 使用移至前端策略处理访问序列 σ 的代价 $C_{MTF}(S)$ 至多是用其他任何启发式策略 H 处理 S 的代价 $C_H(S)$ 的**4**倍。

定理证明



证明： 设 L_i 为第 i 次访问后的MTF表， L_i^* 为第 i 次访问后的最优表；

设 C_i 为MTF表第 i 个操作的代价， C_i^* 为最优表第 i 个操作的代价；

当 x 被访问到时， $c_i = 2 \times \text{rank}_{L_{i-1}}(x)$ ， 因为我们是在 L_{i-1} 里访问到的；

$$c_i^* = \text{rank}_{L_{i-1}^*}(x) + t_i^*$$

t_i^* 指最优表做了 t_i^* 次替换

定理证明



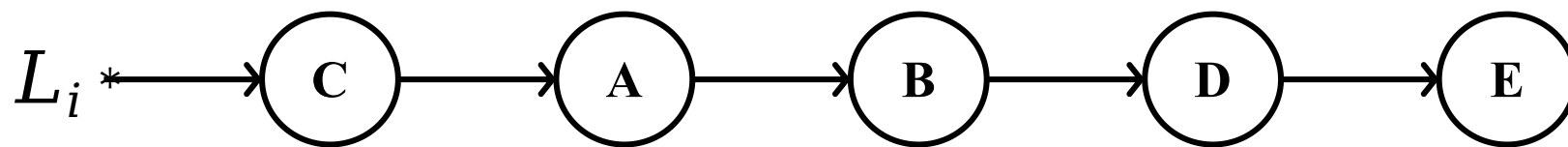
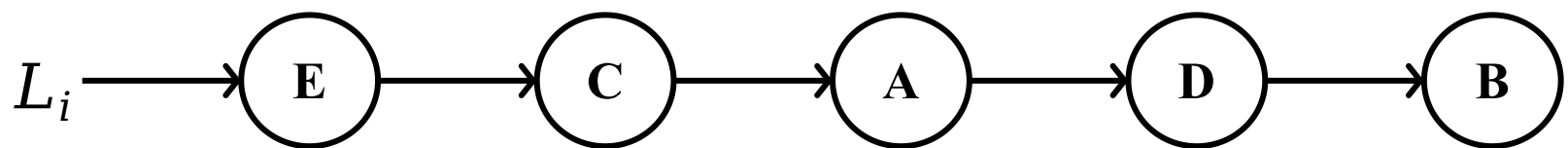
我们定义逆序(inversion)关系: L_i 中一对元素 y 和 z , 在 L_i 中 y 在 z 之前, 在 L_i^* 中 z 在 y 之前。假定处理完访问序列 $\langle s_1, s_2, \dots, s_i \rangle$, L_i 中有 q_i 个逆序关系。然后, 我们定义一个势函数 Φ , 将 L_i 映射到实数

$$\Phi(L_i) = 2q_i。$$

定理证明



例如, 如果 L_i 有元素 $\langle e, c, a, d, b \rangle$, 而 L_i^* 有元素 $\langle c, a, b, d, e \rangle$



那么 L_i 有5个逆序 $((e,c),(e,a),(e,d),(e,b),(d,b))$ 。
 $\Phi(L_i) = 10$

定理证明



对所有 i , 都有 $\Phi(L_i) \geq 0$,

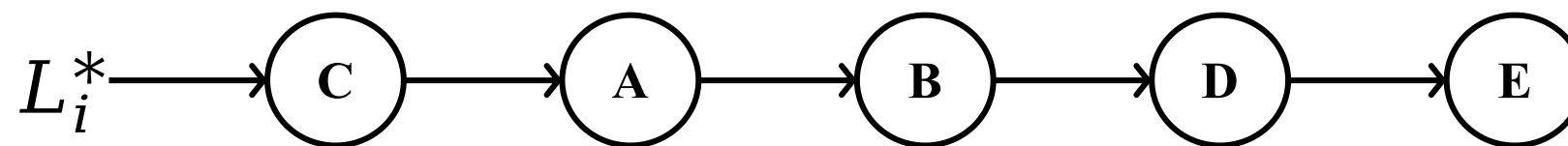
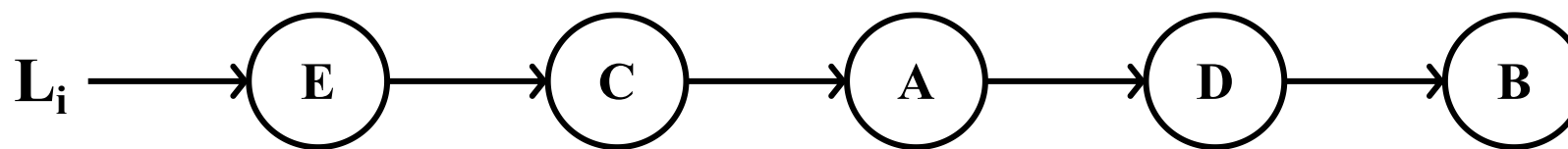
并且如果MTF策略和策略 H 从相同的列表 L_0 开始, 那么 $\Phi(L_0) = 0$ 。

定理证明



一次转置操作会让 Φ 变化多少?

也就是说逆序对的数目会怎样改变?



交换相邻元素会产生或消除一个逆序对, 所以 $\Delta\Phi = \pm 2$

定理证明



假定访问 σ_i 查找元素 x 。为了理解势是如何根据 σ_i 来变化的，我们将除 x 之外的元素划分为4个集合，划分的依据是在第 i 次访问之前它们在列表中的位置：

- 集合A包含在 L_{i-1} 和 L_{i-1}^* 中都位于 x 之前的元素。
- 集合B包含在 L_{i-1} 中位于 x 之前的元素，在 L_{i-1}^* 中位于 x 之后的元素。
- 集合C包含在 L_{i-1} 中位于 x 之后的元素，在 L_{i-1}^* 中位于 x 之前的元素。
- 集合D包含在 L_{i-1} 和 L_{i-1}^* 中都位于 x 之后的元素。
- r 指元素 x 在表中的排位。

定理证明



L_{i-1}	AUB	x	CUD
-----------	------------	----------	------------

L_{i-1}^*	AUC	x	BUD
-------------	------------	----------	------------

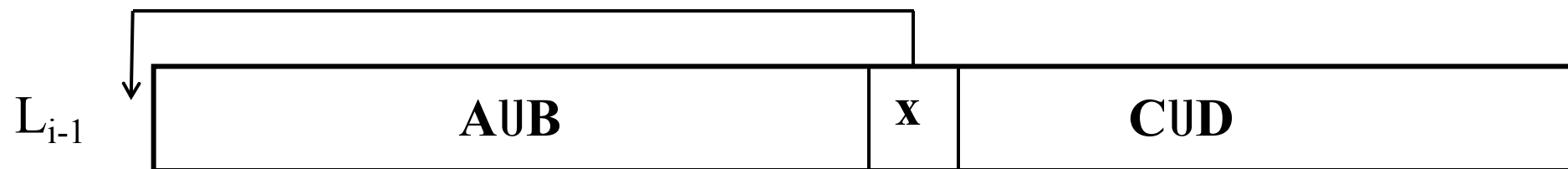
$$r = \text{rank}_{L_{i-1}}(x) = |A| + |B| + 1$$

$$r^* = \text{rank}_{L_{i-1}^*}(x) = |A| + |C| + 1$$

定理证明



操作*i*访问两个列表的*x*时会发生什么？



*x*前移后，最坏情况下A里的每一个元素都会产生一对逆序，B里的所有逆序都会被消除，所以有势能变化：

$$\Phi(L_i) - \Phi(L_{i-1}) \leq 2(|A| - |B| + t_i^*)$$

t_i^* 表示用启发式策略H处理访问 σ_i 期间执行的转置操作次数。

定理证明



所以最优算法也并不比MTF算法好很多。

我们研究第*i*次操作的摊还代价。定义处理访问 σ_i 的摊还代价为

$$\begin{aligned}\widehat{C}_i &= C_i + \Phi(L_i) - \Phi(L_{i-1}) \\ &\leq 2r + 2(|A| - |B| + t_i^*) \\ &= 2r + 2(|A| - (r - |A| - 1) + t_i^*) \\ &= 2r + 4|A| - 2r + 2 + 2t_i^* \\ &\leq 4(r^* + t_i^*) \\ &= 4C_i^*\end{aligned}$$

$$4|A|+2 < 4(|A|+1) \leq 4r^*$$

$$r^* = \text{rank}_{L_{i-1}^*}(x) = |A| + |C| + 1$$

$$2t_i^* < 4t_i^*$$

定理证明



最后我们分析总代价，总代价等于所有单步代价的总和。

$$C_{MTF}(S) = \sum_{i=1}^{|S|} C_i = \sum_{i=1}^{|S|} (\widehat{C}_i + \Phi(L_{i-1}) - \Phi(L_i))$$

$$\leq \sum_{i=1}^{|S|} 4C_i^* + \Phi(L_0) - \Phi(L_{|S|}) \leq 4C_{opt}(S)$$



湖南大学
HUNAN UNIVERSITY

谢谢观赏

—— 实事求是 敢为人先 ——